Closing the Rounding Gap in Differentiable Constraint Programming with Augmented Lagrangian Dual Updates

Vijay Daita Hayley Antczak Miro Swisher Rick Underwood

Stanford University Stanford University Stanford University Independent

ABSTRACT

Differentiable relaxations of combinatorial constraints — where discrete variables are replaced with doubly stochastic matrices projected via Sinkhorn normalization — reliably reach zero loss in the continuous domain. The problem is discretization: Hungarian rounding of the continuous solution fails to satisfy hard constraints at $N \geq 64$, leaving a persistent "rounding gap" between the continuous optimum and a feasible integer solution. We diagnose this failure, characterize it empirically on N-Queens at scales $N \in \{64, 128, 256, 512\}$, and show that the gap closes under an Augmented Lagrangian formulation with per-constraint learnable dual variables. The dual update rule — $\mu \leftarrow \mu + \rho \cdot g(x)$ after each inner optimization — identifies which diagonal constraints are chronically violated and tightens their enforcement until Hungarian rounding produces a feasible solution. Results: 0 violations at $N = 64$ (2.5 s), $N = 128$ (28 s), $N = 256$ (176 s), and $N = 512$ via hybrid ALM + greedy repair (34 min). On Apple M4 Max (MPS), GPU compute crosses over CPU at $N = 16$, reaching 5× speedup at $N = 64$. A warm-start pathway — supplying the continuous solution as a hint to CP-SAT — produces feasible solutions in 100% of trials.

1. Introduction

Sinkhorn-based differentiable solvers for AllDifferent constraints converge reliably in the continuous domain. Across every tested problem size — $N = 8$ through $N = 512$ — the continuous loss reaches exactly 0.000000. The doubly stochastic matrix $P \in DS_N$ produced by Sinkhorn normalization satisfies all row-sum and column-sum constraints to numerical precision. The problem appears at discretization. Hungarian rounding of the converged continuous solution produces integer assignments that violate hard diagonal constraints. At $N = 32$, rounding fails in 1 of 3 trials. From $N \geq 64$ onward, it fails universally: 6 diagonal violations at $N = 64$, 8 at $N = 128$, and worse beyond. Zero continuous loss does not imply a feasible discrete solution. The gap between the continuous optimum and the nearest feasible integer assignment — the *rounding gap* — persists at every scale where it appears, and closing it is the central question of this work.

The rounding gap matters because differentiable programming for combinatorial optimization enables gradient-based solvers on GPU hardware. This is a fundamentally parallel computation model. Sequential constraint programming solvers — CP-SAT [Perron & Furnon 2023], Gecode, OR-Tools — exploit clause learning, propagation, and large neighborhood search, but they do not exploit data parallelism across thousands of arithmetic units. If the rounding gap closes reli-

ably, differentiable CP becomes a viable parallel alternative for problem families where continuous relaxations carry structural information about feasible solutions. The broader differentiable combinatorial optimization literature demonstrates the potential: Erdos Goes Neural [Karalias & Loukas 2020] learns graph combinatorial surrogates end-to-end, Gumbel-Sinkhorn networks [Mena et al. 2018a] produce differentiable permutation samples, SATNet extends differentiable relaxation to satisfiability [Wang et al. 2019], and OptNet applies it to integer linear optimization [Amos & Kolter 2017]. All of these methods face discretization as the final bottleneck.

Prior approaches do not close the rounding gap. Gumbel-Sinkhorn sampling [Mena et al. 2018a] adds stochastic diversity to the rounding step by injecting Gumbel noise into the log-assignment matrix before Sinkhorn normalization. The method treats the rounding problem as a sampling problem: generate many perturbed candidates, round each one, keep the best. At $N = 64$, 500 Gumbel samples all fail — every perturbation of the continuous optimum rounds to a permutation that violates at least one diagonal constraint. The problem is not a lack of diversity in rounding candidates. The problem is the *location* of the continuous optimum itself. It sits in a region of the Birkhoff polytope where all nearby vertices (permutation matrices) are infeasible. Fixed-penalty methods can increase the constraint weight λ to push the optimum elsewhere, but large fixed penalties degrade the conditioning of the optimization landscape and slow inner-loop convergence. The fundamental issue is that the continuous objective is blind to which specific constraints will be violated after rounding, and a fixed penalty cannot adapt to the violation pattern of a particular problem instance.

Augmented Lagrangian methods [Bertsekas 1999; Nocedal & Wright 2006] address exactly this limitation. ALM maintains per-constraint dual variables μ_k that accumulate based on observed violations. After each inner optimization, the update rule applies: $\mu_k \leftarrow \mu_k + \rho \cdot g_k(x^*)$, where $g_k(x^*)$ measures the violation of constraint k at the current solution. Constraints that remain violated after rounding receive growing Lagrange multipliers. Constraints that are satisfied receive no additional pressure. The dual variables steer the continuous optimum toward a region of the Birkhoff polytope where Hungarian rounding succeeds. Two to three outer iterations suffice at all tested scales ($N \in \{64, 128, 256, 512\}$). The dual variables learn problem-specific structure: diagonal constraints of different lengths have different violation propensities, and the ALM update weights them accordingly without any prior knowledge of the problem geometry.

The contributions of this paper are four. First, an empirical diagnosis of the rounding gap establishes that it is a function of problem size N, is not closed by Gumbel-Sinkhorn sampling (500 samples, all infeasible at $N = 64$), and grows sublinearly with N. Second, an ALM formulation with per-diagonal dual variables closes the gap: 0 violations at $N = 64$ (2.5 s), $N = 128$ (28 s), $N = 256$ (176 s), and $N = 512$ via hybrid ALM + greedy repair (34 min). Third, a GPU crossover analysis on Apple MPS hardware quantifies when batched Sinkhorn iterations become faster than CPU execution: crossover at $N = 16$, peak speedup of 4.95× at $N = 64$. Fourth, a warm-start pathway — supplying the continuous solution as a hint to CP-SAT — produces feasible solutions in 100% of trials across all tested sizes, including $N = 64$ and $N = 128$ where standalone rounding fails.

2. Background and Related Work

Sinkhorn normalization and the Birkhoff polytope. Sinkhorn [1964] proved that any square matrix with strictly positive entries converges to a doubly stochastic matrix under alternating row and column normalizations. Birkhoff [1946] established that the set DS_N of $N \times N$ doubly stochastic matrices is the convex hull of

the $N!$ permutation matrices — the Birkhoff polytope. Together these results provide the geometric foundation for differentiable constraint programming: Sinkhorn iteration projects an arbitrary log-assignment matrix onto the continuous relaxation of the set of feasible integer assignments. The iteration operates as follows: given a matrix A, set $S_0 = exp(A / \tau)$ for temperature $\tau > 0$, then alternate row normalization $S_{t+1/2} = D_r^{-1} S_t$ and column normalization $S_{t+1} = S_{t+1/2} D_c^{-1}$, where D_r and D_c are diagonal matrices of row and column sums respectively. Each iteration costs $O(N^2)$ and consists entirely of matrix operations — all of which map directly to GPU execution.

Differentiable sorting and ranking. Adams and Zemel [2011] first applied Sinkhorn normalization to ranking problems, showing that a differentiable relaxation of the permutation operator enables gradient-based learning-to-rank. Cuturi [2013] connected Sinkhorn iterations to entropy-regularized optimal transport, establishing a dual interpretation: the regularization parameter ε plays the same role as the temperature τ. Mena et al. [2018a] introduced the Gumbel-Sinkhorn network, which adds Gumbel noise to the log-assignment matrix before Sinkhorn normalization to produce stochastic, differentiable permutation samples — the rounding strategy evaluated and found insufficient in this paper. Mena et al. [2018b] applied Sinkhorn networks to learning permutations end-to-end in supervised settings. These papers establish the direction of differentiable permutation operators. The present work identifies where this direction breaks down: at the rounding boundary between continuous and discrete solutions.

Differentiable combinatorial optimization. Karalias and Loukas [2020] proposed Erdos Goes Neural, a framework that trains a graph neural network to minimize a differentiable surrogate of a combinatorial objective (maximum independent set, minimum dominating set, graph coloring). The surrogate loss reaches near-zero in the continuous domain; the final step is rounding to a discrete solution. Wang et al. [2019] extend differentiable relaxation to Boolean satisfiability (SATNet), and Amos and Kolter [2017] apply similar principles to differentiable quadratic optimization (OptNet). These methods share a common architecture: relax the discrete domain, optimize the continuous surrogate with gradient descent, round the result. All of them face the rounding gap, and none address it through adaptive dual-variable updates. Standard remedies are either larger fixed penalties (which degrade conditioning) or repeated sampling (which fails when the continuous optimum lies in an infeasible region of the relaxation polytope).

Augmented Lagrangian methods. The Augmented Lagrangian Method was introduced independently by Hestenes [1969] and Powell [1969]. The method augments a constrained optimization problem $min\ f(x)\ s.t.\ g(x) \leq 0$ by adding both a linear dual term and a quadratic penalty term to the objective: $\mathcal{L}(x, \mu, \rho) = f(x) + \mu^T g(x) + (\rho/2) \|g^+(x)\|^2$. The dual update $\mu \leftarrow \mu + \rho \cdot g^+(x^*)$ after each inner minimization converts the history of constraint violations into a steering signal. Bertsekas [1999, Chapter 4] provides the convergence theory: under mild regularity conditions, ALM converges to a KKT point without requiring ρ to grow unboundedly. Nocedal and Wright [2006, Chapter 17] emphasize that ALM maintains good conditioning because the dual term carries the accumulated violation pressure. This distinguishes ALM from pure penalty methods, where ρ must grow large to enforce constraints, destroying the conditioning of the inner problem. The present work applies ALM in a batched, GPU-native formulation where each of B parallel restarts carries its own vector of dual variables.

Constraint programming and CP-SAT. CP-SAT [Perron & Furnon 2023] is the reference modern constraint programming solver. It combines clause-driven conflict learning, constraint propagation, large neighborhood search, and LP

relaxations. On N-Queens — the benchmark problem used throughout this paper — CP-SAT solves $N = 64$ in 89 ms and $N = 128$ in 463 ms (single-threaded, median of 3 trials). DiffCP does not compete with CP-SAT on raw sequential speed at any scale tested. The value proposition is different: GPU scalability for the Sinkhorn inner loop (which achieves 4.95× speedup at $N = 64$ on Apple MPS) and the warm-start pathway, where the continuous solution provides an integer hint that CP-SAT accepts and uses to seed its search. The Hungarian algorithm [Kuhn 1955] — the rounding step that converts a doubly stochastic matrix to a permutation — runs in $O(N^3)$ on CPU after each inner optimization.

No prior work combines batched Sinkhorn optimization over B parallel restarts, Augmented Lagrangian dual updates per restart with per-constraint granularity, GPU-native execution on Apple MPS or NVIDIA CUDA, and a warm-start pathway to a production CP solver. Each component exists independently; the combination and the empirical diagnosis of the rounding gap are the contributions of this paper.

3. Method

Problem formulation. The goal is to find an assignment $x \in \{0, \ldots, N-1\}^N$ satisfying a set of constraints $C = \{c_1, \ldots, c_m\}$. For permutation constraint satisfaction problems, the AllDifferent constraint requires x to be a bijection: each value appears exactly once. N-Queens imposes two additional families of constraints beyond AllDifferent. The forward diagonal constraint requires all values $i - x_i$ to be distinct for $i = 0, \ldots, N-1$. The backward diagonal constraint requires all values $i + x_i$ to be distinct. Define the assignment matrix $X \in \{0,1\}^{N \times N}$, where $X_{ij} = 1$ if and only if $x_i = j$. A feasible N-Queens solution corresponds to a permutation matrix X in which no two entries equal to 1 share a forward or backward diagonal.

Sinkhorn relaxation. The discrete optimization over permutation matrices is relaxed to a continuous optimization over the Birkhoff polytope DS_N:

$$DS_N = \{ P \in \mathbb{R}^{N \times N} : P_{ij} \geq 0,\ \textstyle\sum_j P_{ij} = 1\ \forall i,\ \sum_i P_{ij} = 1\ \forall j \}. \tag{1}$$

The Birkhoff–von Neumann theorem states that the vertices of DS_N are exactly the $N!$ permutation matrices. Any doubly stochastic matrix is therefore a convex combination of permutations. The Sinkhorn operator $S(\log \alpha, \tau)$ projects an arbitrary log-assignment matrix $\log \alpha \in \mathbb{R}^{N \times N}$ onto DS_N via K iterations of alternating log-space row and column normalization at temperature τ:

$$\log P^{(t+1/2)} = \log P^{(t)} - LSE_{row}(\log P^{(t)}),$$
$$\log P^{(t+1)} = \log P^{(t+1/2)} - LSE_{col}(\log P^{(t+1/2)}),$$

where $\log P^{(0)} = \log \alpha / \tau$ and $\mathrm{LSE}_{\mathrm{row}}$ denotes the log-sum-exp computed along each row (analogously for columns). Each iteration costs $O(N^2)$ and consists entirely of element-wise and reduction operations that map directly to GPU execution. We use $K = 20$ Sinkhorn iterations throughout.

Temperature schedule. The temperature τ controls the sharpness of P. At high τ, the Sinkhorn output is a uniform doubly stochastic matrix. As $\tau \to 0$, P approaches a permutation matrix. The temperature follows an exponential decay schedule:

$$\tau_t = \tau_0 \cdot (\tau_T / \tau_0)^{t/T}, \tag{2}$$

where $\tau_0 = 2.0$ is the initial temperature, $\tau_T = 0.01$ is the final temperature, and T is the number of Adam inner iterations. At the beginning of each ALM outer iteration, the starting temperature resets to $\tau_0 \cdot (1 - 0.5 \cdot o/O)$, where o is the current outer iteration index and O is the

total number of outer iterations. This ensures that each successive outer round begins at a slightly lower temperature, reflecting the accumulated dual signal.

Constraint surrogates. Sinkhorn normalization enforces AllDifferent on rows and columns by construction: every output of the Sinkhorn operator is doubly stochastic. The diagonal constraints require separate enforcement. Define the forward and backward diagonal sums via scatter-add:

$$d^+_k(P) = \sum_{i-j=k} P_{ij}, \quad d^-_k(P) = \sum_{i+j=k} P_{ij},$$

where $k \in \{-(N-1), \ldots, N-1\}$ for forward diagonals and $k \in \{0, \ldots, 2N-2\}$ for backward diagonals. For a feasible N-Queens assignment, each diagonal sum equals either 0 or 1. The constraints are $d^+_k \leq 1$ and $d^-_k \leq 1$ for all k. The standard differentiable surrogate penalizes violations quadratically:

$$L(P) = \sum_k ReLU(d^+_k(P) - 1)^2 + \sum_k ReLU(d^-_k(P) - 1)^2.$$

Batch formulation. The solver runs B parallel restarts to increase the probability of finding a feasible rounding. Initialize $\log \alpha \in \mathbb{R}^{B \times N \times N}$ with entries drawn from $\mathcal{N}(0, 0.01)$. Each restart $b \in \{1, \ldots, B\}$ produces a doubly stochastic matrix $P^{(b)} = S(\log \alpha^{(b)}, \tau_t)$ and incurs a per-restart loss $\ell^{(b)} = L(P^{(b)})$. The total loss $\sum_b \ell^{(b)}$ is minimized by the Adam optimizer [Kingma & Ba 2015] with learning rate 0.2. At termination, the restart with the lowest loss is selected and rounded. This batch formulation maps to a single (B, N, N) tensor operation on GPU:

$$P^{(b)} = S(\log \alpha^{(b)}, \tau_t), \quad \ell^{(b)} = L(P^{(b)}), \quad \mathcal{L}_{total} = \sum_{b=1}^{B} \ell^{(b)}. \tag{3}$$

Rounding and the gap. After inner optimization completes, the best restart is identified: $P^* = \mathrm{argmin}_b\, \ell^{(b)}$, evaluated at $\tau = \tau_T$. The Hungarian rounding operator H computes the minimum-cost assignment on the negated matrix $-P^*$ via the Kuhn–Munkres algorithm [Kuhn 1955], yielding a discrete assignment $x = H(P^*)$. This assignment is a permutation by construction and therefore satisfies AllDifferent. It does not, in general, satisfy the diagonal constraints. Define the violation count $V(x)$ as the number of diagonal constraints that the discrete solution violates. For $N \leq 32$, $V(x) = 0$ consistently across all trials. For $N \geq 64$, $V(x) \geq 6$ in every trial. This is the rounding gap: the continuous loss is zero, but the discrete solution is infeasible. The gap arises because the continuous optimum lies in a region of the Birkhoff polytope where all nearby permutation-matrix vertices violate at least one diagonal constraint.

Augmented Lagrangian formulation. The ALM replaces the fixed-penalty surrogate with an adaptive loss that maintains per-constraint dual variables. Define the constraint violation functions:

$$g^+_k(P) = ReLU(d^+_k(P) - 1), \quad g^-_k(P) = ReLU(d^-_k(P) - 1).$$

Introduce dual variables $\mu^+_k, \mu^-_k \in \mathbb{R}$ for each diagonal k and each restart (suppressing the restart index for clarity). The ALM loss for one outer iteration is:

$$L_{ALM}(P; \mu, \rho) = \sum_k [\mu^+_k \cdot g^+_k(P) + (\rho/2)(g^+_k(P))^2] + \sum_k [\mu^-_k \cdot g^-_k(P) + (\rho/2)(g^-_k(P))^2]. \tag{4}$$

The inner loop minimizes L_{ALM} over $\log \alpha$ with Adam for T_{inner} steps at the current values of μ and ρ. The linear terms $\mu^+_k \cdot g^+_k$ bias the gradient toward reducing violations on diagonals that were violated in previous outer iterations. Because μ carries the accumulated violation history, ρ need not grow large to enforce feasibility — this preserves the conditioning of the inner problem, unlike a pure penalty method.

Dual update rule. After each inner optimization converges, the dual variables and penalty coefficient are updated:

$$\mu^+_k \leftarrow \mu^+_k + \rho \cdot g^+_k(P^*), \quad \mu^-_k \leftarrow \mu^-_k + \rho \cdot g^-_k(P^*), \tag{5}$$

$$\rho \leftarrow \min(\rho \cdot \beta, \rho_{max}),$$

where $\beta = 2.0$, $\rho_{max} = 200$, and $\rho_{init} = 5.0$. Diagonals with zero violation after the inner solve receive no update to their dual variable. Diagonals with persistent violation accumulate increasing μ values across outer iterations, which focuses the gradient signal on the hardest constraints. After each dual update, Hungarian rounding is attempted on all B restarts in order of increasing violation magnitude. If any restart yields $V(x) = 0$, the algorithm returns immediately with a feasible solution.

Algorithm 1: ALM Solver for Permutation CSPs

Input: *Board size N, restarts B, outer iterations O, inner iterations T, Sinkhorn iterations K, learning rate η, temperatures τ_0, τ_T, penalty schedule ρ_{init}, β, ρ_{max}.*
Output: *Feasible assignment $x \in \{0, \ldots, N-1\}^N$ or best-effort solution.*

1. Initialize $\log \alpha \sim N(0, 0.01)^{B \times N \times N}$; set $\mu^+_k \leftarrow 0$, $\mu^-_k \leftarrow 0$ for all diagonals k; set $\rho \leftarrow \rho_{init}$.
2. **For** $o = 1, \ldots, O$:
   a. Set $\tau_{start} \leftarrow \tau_0 \cdot (1 - 0.5 \cdot o/O)$.
   b. Initialize Adam optimizer on $\log \alpha$ with learning rate η.
   c. **For** $t = 1, \ldots, T$:
      i. Compute temperature: $\tau_t \leftarrow \tau_{start} \cdot (\tau_T / \tau_{start})^{t/T}$.
      ii. Compute $P^{(b)} \leftarrow S(\log \alpha^{(b)}, \tau_t)$ for all b via K Sinkhorn iterations.
      iii. Compute diagonal violations $g^+_k(P^{(b)})$, $g^-_k(P^{(b)})$ via scatter-add.
      iv. Compute $L_{ALM}(P; \mu, \rho)$ per Equation (4).
      v. Backpropagate and update $\log \alpha$ via Adam.
   d. Evaluate $P^{(b)} \leftarrow S(\log \alpha^{(b)}, \tau_T)$ at final temperature for all b.
   e. Compute violations $g^+_k(P^{(b)})$, $g^-_k(P^{(b)})$ at τ_T.
   f. Update dual variables: $\mu^+_k \leftarrow \mu^+_k + \rho \cdot g^+_k$, $\mu^-_k \leftarrow \mu^-_k + \rho \cdot g^-_k$.
   g. Update penalty: $\rho \leftarrow \min(\rho \cdot \beta, \rho_{max})$.
   h. **For** each restart b in order of increasing $\sum_k (g^+_k)^2 + (g^-_k)^2$:
      i. Compute $x^{(b)} \leftarrow H(P^{(b)})$ via Hungarian algorithm.
      ii. **If** $V(x^{(b)}) = 0$: **return** $x^{(b)}$.
3. **Return** $x^{(b^*)}$ where $b^* = \text{argmin}_b V(x^{(b)})$ (best-effort).

Greedy repair for large *N*. At $N = 512$, pure ALM reduces the violation count to a small residual (typically $V(x) \leq 10$) but does not close to zero within a practical number of outer iterations. A hybrid strategy combines ALM with greedy min-conflicts repair [Minton et al. 1992]. After ALM terminates, the repair phase proposes random pairwise queen swaps: select two rows i, j uniformly at random and swap $x_i \leftrightarrow x_j$. Accept the swap if and only if it reduces the total conflict count. The swap preserves the permutation structure, so AllDifferent remains satisfied throughout. With the ALM warm start, greedy repair closes the remaining gap in fewer than 5000 swap attempts for $N = 512$.

Warm-start pathway. Independent of whether ALM closes the rounding gap entirely, the continuous solution P^* encodes structural information about feasible solutions. CP-SAT [Perron & Furnon 2023] accepts solution hints: for each variable x_i, the hint value is set to $argmax_j P^*_{ij}$. The hint construction is $O(N)$ — a single argmax per row. In all 15 trials across $N \in \{8, 16, 32, 64, 128\}$, CP-SAT finds a feasible solution when initialized with this hint, achieving a 100% success rate. The warm-start pathway is useful when feasibility guarantees are required and the latency overhead of running a CP solver after the differentiable phase is acceptable.

Complexity. Each ALM outer iteration performs T inner Adam steps, each requiring K Sinkhorn iterations on a (B, N, N) tensor. The per-step cost is $O(B \cdot K \cdot N^2)$ for Sinkhorn normalization plus $O(B \cdot N^2)$ for the scatter-add diagonal computation and backward pass. The total inner loop cost per outer iteration is $O(B \cdot T \cdot K \cdot N^2)$. Hungarian rounding after each outer iteration costs $O(B \cdot N^3)$ for B independent assignments. Memory is $O(B \cdot N^2)$ for the log-assignment tensor and the doubly stochastic matrices, plus $O(B \cdot N)$ for the dual variables.

Figure 1

Architecture and data-flow diagram showing the three phases of the DiffCP pipeline. Phase 1 (top): log α (B×N×N) → Sinkhorn(·, τ_t) → P (doubly stochastic) → DiagonalConflictLoss → Adam backward. Phase 2 (middle): ALM outer loop with μ update. Phase 3 (bottom): rounding branch H(P) and warm-start branch argmax P* → CP-SAT hint.*

Figure 1: Each of $B = 64$ restarts runs independently with its own dual variables. Sinkhorn normalization enforces AllDifferent; diagonal penalty enforces the N-Queens structural constraints. The warm-start branch (right) operates independently of the ALM rounding branch (left).

4. Experiments

Setup. All experiments at $N \leq 256$ ran on an Apple M4 Max with a 10-core ARM CPU and a 40-core GPU via Metal Performance Shaders (MPS) in PyTorch 2.x. The $N = 512$ experiment ran on an Apple M3 Ultra with a 32-core CPU, 76-core MPS GPU, and 256 GB unified memory. The benchmark problem is N-Queens at $N \in \{8, 16, 32, 64, 128, 256, 512\}$. The CP-SAT baseline uses Google OR-Tools v9.x, single-threaded, with a 120-second time limit. All reported times are wall-clock medians of 3 trials. Batch size $B = 64$ for all DiffCP runs. Inner iterations $T = 1500$, Sinkhorn iterations $K = 20$, learning rate $\eta = 0.2$, initial temperature $\tau_0 = 2.0$, final temperature $\tau_T = 0.01$, penalty parameters $\rho_{init} = 5.0$, $\beta = 2.0$, $\rho_{max} = 200$.

Experiment 1: Rounding gap diagnosis. Table 1 presents the core diagnostic result. The continuous loss reaches 0.000000 at every tested N — the Sinkhorn-optimized doubly stochastic matrix P satisfies all diagonal surrogates to numerical precision. The gap appears at discretization. Hungarian rounding produces zero violations at $N = 32$ in 2 of 3 trials but fails outright at $N \geq 64$. The violation count $V(x)$ grows sublinearly: 6 at $N = 64$, 8 at $N = 128$, and beyond 10 at $N = 512$. This sublinear growth is informative: violations concentrate on the long central diagonals. ALM closes the gap at every N: zero violations at $N \leq 256$ in 2–3 outer iterations, and zero violations at $N = 512$ after 10 ALM outer iterations followed by greedy repair.

Table 1. Rounding gap diagnosis across problem sizes. Continuous loss is the final $L(P)$ before rounding. Hungarian violations report $V(x)$ after Hungarian rounding $H(P^*)$; parenthetical fractions give trials (out of 3) achieving $V = 0$. ALM violations report $V(x)$ after the full ALM pipeline.

N	Continuous Loss	Hungarian $V(x)$	ALM $V(x)$	ALM Outer Iters
32	0.000000	0 (2/3 trials)	**0**	2
64	0.000000	6 (0/3)	**0**	2
128	0.000000	8 (0/3)	**0**	3
256	0.000000	>8 (0/3)	**0**	3
512	0.000000	>10 (0/3)	**0 (ALM + repair)**	10 + repair

Gumbel-Sinkhorn negative result. To isolate whether the rounding gap is a sampling problem or a location problem, we applied Gumbel-Sinkhorn rounding at $N = 64$: 500 candidates sampled with Gumbel noise at $\tau = 0.05$ and $K = 30$ Sinkhorn iterations per sample. All 500 candidates violated at least one diagonal constraint — zero feasible solutions out of 500 attempts. Gumbel-Sinkhorn diversifies the rounding step by sampling different permutation-matrix vertices near the continuous optimum. Its universal failure at $N = 64$ demonstrates that the problem is not rounding diversity: every vertex in the neighborhood of the converged P^* is infeasible. ALM addresses this by moving the optimum itself, not by sampling more aggressively around a fixed optimum.

Figure 4

Grouped bar chart: rounding strategy comparison at N = 64. Strategies (y-axis): Standard Hungarian, Gumbel-500, ALM. Bars show diagonal violations after rounding. ALM bar reaches 0 (highlighted). Gumbel-500 matches Standard Hungarian, confirming the location hypothesis.

Figure 4: At $N = 64$, only the Augmented Lagrangian solver produces 0 violations. Gumbel-Sinkhorn with 500 samples matches the standard Hungarian result, confirming that the issue is the location of the continuous optimum rather than the rounding procedure itself.

Experiment 2: ALM convergence. The ALM closes the rounding gap in 2–3 outer iterations at all $N \leq 256$. The mechanism is visible in the dual variable trajectory. At initialization, all $\mu^+_k = \mu^-_k = 0$. After outer iteration 1, the dual variables are nonzero only on diagonals that were violated after rounding. The long central diagonals — those with indices k near 0 for forward diagonals and near N for backward diagonals — accumulate the largest μ values. These diagonals span the most board positions and are the most contested. By outer iteration 2, the accumulated dual signal is sufficient to steer the continuous optimum into a region where Hungarian rounding succeeds. The geometric penalty schedule $\rho_{new} = min(\rho \cdot 2.0, 200)$ prevents conditioning collapse: ρ reaches 20 after two outer iterations, far below the $\rho_{max} = 200$ cap. At $N = 512$, the ALM requires 10 outer iterations to reduce $V(x)$ to a small residual ($V \leq 10$), after which greedy repair closes the remaining violations in fewer than 5000 swap attempts.

Figure 2

Left panel: per-outer-iteration inner loss curve for N = 64 (Adam steps vs. L_ALM loss, three curves for outer iters 1–3, dashed vertical at first successful rounding). Right panel: dual variable L2 norm ‖μ‖ vs. outer iteration for N ∈ {64, 128, 256}, showing exponential growth until rounding succeeds.

Figure 2: Dual variables accumulate violation pressure across outer iterations. Rounding succeeds when $\|\mu\|$ reaches a problem-specific threshold, typically by outer iteration 2–3 at $N \leq 256$.

Experiment 3: Scaling. Table 2 compares DiffCP against CP-SAT on sequential wall-clock time. DiffCP with standard (non-ALM) penalty optimization is slower than CP-SAT at every tested N. At $N = 64$, CP-SAT solves in 89 ms; DiffCP standard takes 32.8 s — a factor of 367× slower — and fails to produce a feasible solution. With ALM (Table 3), DiffCP achieves feasibility but remains 28× slower than CP-SAT at $N = 64$ (2.5 s vs. 89 ms). DiffCP is not competitive with CP-SAT on sequential hardware at any tested scale. The contribution is the ALM rounding result — the demonstration that the gap closes reliably — and the GPU pathway that this enables.

Table 2. DiffCP standard (fixed penalty, no ALM) vs. CP-SAT. Feasibility indicates how many of 3 trials produced $V(x) = 0$. Warm-start feasibility indicates how many trials produced a feasible CP-SAT solution seeded with the DiffCP hint.

N	CP-SAT (ms)	DiffCP Standard (ms)	DiffCP Feas.	Warm-Start Feas.
8	**3.4**	143	3/3	**3/3**
16	**7.1**	1,746	3/3	**3/3**
32	**20.5**	14,773	1/3	**3/3**
64	**89.4**	32,786	0/3	**3/3**
128	**463**	119,327	0/3	**3/3**

Table 3. ALM results on MPS GPU. Time includes all outer iterations and Hungarian rounding. The $N = 512$ entry includes greedy repair after ALM.

N	ALM Time	Outer Iters	Device
64	2.5 s	2	M4 Max MPS
128	28 s	3	M4 Max MPS
256	176 s	3	M4 Max MPS
512	34 min	10 + repair	M3 Ultra MPS

Experiment 4: GPU vs. CPU crossover. Table 4 reports the inner-loop execution time for a single ALM outer iteration on CPU versus MPS GPU. The crossover occurs at $N = 16$, where MPS matches CPU execution time. At $N = 64$, MPS is 4.95× faster than CPU. At $N = 8$, MPS is 0.17× — a 5.9× slowdown — due to MPS kernel launch overhead dominating the small matrix operations. At $N = 128$, the speedup drops to 1.89×, a regression from the $N = 64$ peak. Profiling attributes this to MPS memory bandwidth saturation: the *(64, 128, 128)* batch tensor is 8.4 MB, and the 20 Sinkhorn iterations per Adam step generate sufficient memory traffic to saturate the M4 Max's unified memory bus. The peak speedup at $N = 64$ represents the sweet spot where the *(64, 64, 64)* batch tensor (2.1 MB) fits comfortably in the GPU's working set and the

arithmetic intensity of Sinkhorn normalization is high enough to amortize kernel launch overhead.

Table 4. Inner-loop execution time for one ALM outer iteration, CPU vs. MPS GPU, with $B = 64$ restarts.

N	CPU (ms)	MPS (ms)	Speedup
8	105	620	0.17×
16	865	806	1.07×
32	5,426	1,504	3.61×
64	24,088	4,861	**4.95×**
128	88,671	46,940	1.89×

Figure 3

Log-scale line plot of wall-clock time (ms) vs. N for CPU (blue) and MPS (orange). Vertical dashed line at crossover N = 16. Peak speedup annotated at N = 64 (4.95×). Speedup regression at N = 128 labeled "memory bandwidth saturation."

Figure 3: MPS (Apple Silicon GPU) breaks even at $N = 16$ and peaks at 4.95× speedup at $N = 64$. Speedup regresses at $N = 128$, consistent with memory bandwidth saturation on a 40-core GPU.

Ablation: penalty schedule. We varied the penalty multiplier $\beta \in \{1.5, 2.0, 3.0\}$ at $N = 64$ with all other parameters fixed. At $\beta = 2.0$ (the default), the ALM closes the gap in 2 outer iterations. At $\beta = 1.5$, convergence requires 3 outer iterations but the inner optimization is more stable: the final continuous loss is lower before rounding and the dual variable magnitudes are smaller. At $\beta = 3.0$, the ALM also closes in 2 outer iterations, but the inner optimization shows larger gradient variance in the final 100 steps of each inner loop, consistent with the larger ρ degrading conditioning. The ρ_{max} cap has no effect in practice at $N = 64$: with $\beta = 2.0$ and 2 outer iterations, ρ reaches only 20, far below the cap of 200. The cap serves as a safety bound for larger N where more outer iterations are needed.

Limitations. Three limitations bound the generality of these results. First, all experiments use N-Queens exclusively. Extending DiffCP to graph coloring requires dual variables indexed by edges rather than diagonals. Job-shop scheduling requires dual variables indexed by machine-time intervals. These constraint-specific dual variable designs are not validated. Second, DiffCP is not competitive with CP-SAT on wall-clock time at any tested scale. The 4.95× GPU speedup applies to the Sinkhorn inner loop only; the full pipeline — including ALM outer iterations and Hungarian rounding on CPU — is 28× slower than CP-SAT at $N = 64$. Third, memory scales as $O(B \cdot N^2)$: at $N = 512$ with $B = 64$, the batch tensor occupies 67 MB in float32. At $N = 1024$, a batch of $B = 64$ would require 268 MB, necessitating either a reduction in B or a switch to bfloat16 precision. The bfloat16 path has not been tested and may affect Sinkhorn convergence at low τ.

5. Discussion

What the dual variables learn. The dual variable magnitudes after ALM convergence reveal the violation structure of N-Queens. The

long central diagonals — forward diagonals with $|k| \leq N/4$ and backward diagonals with $|k - N| \leq N/4$ — accumulate μ values 3–5× larger than the short corner diagonals. These central diagonals span the most board positions and intersect the most rows, making them the hardest to satisfy simultaneously. The ALM discovers this geometry without any prior encoding: the dual update rule $\mu_k \leftarrow \mu_k + \rho \cdot g_k(P^*)$ simply accumulates violation history, and the violation history reflects the problem structure. This observation suggests a practical acceleration: initialize μ with problem-specific priors proportional to diagonal length rather than at zero. Such warm initialization would reduce the number of outer iterations needed by front-loading the structural signal that ALM currently discovers empirically.

Warm-start as a lower bound on utility. Even when ALM does not close the rounding gap entirely — as at $N = 512$ before repair — the continuous solution P^* contains structural information. The argmax hint $argmax_j\ P^*_{ij}$ fed to CP-SAT produces a feasible solution in 100% of trials across all 15 runs at $N \in \{8, 16, 32, 64, 128\}$. This 100% rate includes $N = 64$ and $N = 128$, where standalone Hungarian rounding fails in every trial. The differentiable solver, despite being slower and less reliable than CP-SAT alone, generates initialization quality that CP-SAT exploits. This is an underexplored direction in the differentiable combinatorial optimization literature: rather than competing with exact solvers, differentiable methods can serve as heuristic generators that provide high-quality starting points. The cost of the hint construction is $O(N)$ — a single argmax per row — and is negligible relative to the solve time of either method.

GPU trajectory. The 4.95× MPS speedup at $N = 64$ is a data point, not a ceiling. Apple MPS on the M4 Max delivers approximately 7 TFLOPS of float32 throughput. An NVIDIA H100 with BF16 Tensor Cores delivers approximately 300 TFLOPS — a 43× ratio in peak arithmetic throughput. The Sinkhorn inner loop is a sequence of batched matrix normalizations with high arithmetic intensity and regular memory access patterns. A conservative projection: the inner loop on H100 would run 30–40× faster than on M4 Max MPS, shifting the crossover with CP-SAT from $N = 16$ to a substantially larger N. The ALM outer loop is embarrassingly parallel across restarts — each of the B restarts carries independent dual variables and requires no inter-restart communication. On multi-GPU hardware, the batch dimension B can scale linearly with GPU count. These projections are not validated experimentally. They indicate the direction and approximate magnitude of the scaling opportunity.

6. Conclusion

The rounding gap in Sinkhorn-based differentiable constraint programming closes under Augmented Lagrangian dual updates with per-diagonal granularity. We diagnosed the gap as a location problem — the continuous optimum sits in a region of the Birkhoff polytope where all nearby permutation-matrix vertices violate at least one diagonal constraint — and showed that Gumbel-Sinkhorn sampling (500 candidates, $\tau = 0.05$) cannot close it. ALM solved N-Queens at $N = 64$ in 2.5 s (2 outer iterations), $N = 128$ in 28 s (3 outer iterations), $N = 256$ in 176 s (3 outer iterations), and $N = 512$ in 34 min (10 outer iterations + greedy repair), achieving zero violations at every scale. GPU execution on Apple MPS accelerated the Sinkhorn inner loop by 4.95× at $N = 64$, with crossover at $N = 16$. The warm-start pathway — feeding the continuous solution as a hint to CP-SAT — produced feasible solutions in all 15 trials across $N \in \{8, 16, 32, 64, 128\}$.

Two open questions remain. First, does ALM generalize beyond permutation CSPs? Graph coloring requires dual variables indexed by edges, bin packing by capacity constraints, and job-shop scheduling by machine-time intervals. Each problem family demands a constraint-specific dual variable design; the ALM framework is

general, but the diagonal-indexed formulation validated here is not. Second, can the warm-start value be quantified systematically? The 100% feasibility rate across all tested sizes establishes that DiffCP hints help CP-SAT, but the magnitude of the acceleration — and the problem characteristics that predict it — remain open. Answering these questions requires experiments on diverse CSP families and a controlled comparison of CP-SAT solve times with and without differentiable warm starts across problem size, constraint density, and solution density.

References

[Adams & Zemel 2011] Adams, R. P., & Zemel, R. S. (2011). Ranking via Sinkhorn Propagation. *arXiv preprint arXiv:1106.1925*.

[Amos & Kolter 2017] Amos, B., & Kolter, J. Z. (2017). OptNet: Differentiable Optimization as a Layer in Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 136–145.

[Bengio et al. 2021] Bengio, Y., Lodi, A., & Prouvost, A. (2021). Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2), 405–421.

[Bertsekas 1999] Bertsekas, D. P. (1999). *Nonlinear Programming*, 2nd ed. Athena Scientific.

[Birkhoff 1946] Birkhoff, G. (1946). Three observations on linear algebra. *Universidad Nacional de Tucumán, Revista (Series A)*, 5, 147–151.

[Cuturi 2013] Cuturi, M. (2013). Sinkhorn Distances: Lightspeed Computation of Optimal Transport Distances. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26.

[Hestenes 1969] Hestenes, M. R. (1969). Multiplier and gradient methods. *Journal of Optimization Theory and Applications*, 4(5), 303–320.

[Karalias & Loukas 2020] Karalias, N., & Loukas, A. (2020). Erdos Goes Neural: An Unsupervised Learning Framework for Combinatorial Optimization on Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33.

[Kingma & Ba 2015] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*.

[Kotary et al. 2021] Kotary, J., Fioretto, F., Van Hentenryck, P., & Wilder, B. (2021). End-to-End Constrained Optimization Learning: A Survey. In *Proceedings of the 30th IJCAI*, Survey Track.

[Kuhn 1955] Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2), 83–97.

[Mena et al. 2018a] Mena, G., Belanger, D., Linderman, S., & Snoek, J. (2018). Learning Latent Permutations with Gumbel-Sinkhorn Networks. In *International Conference on Learning Representations (ICLR)*.

[Mena et al. 2018b] Mena, G., Belanger, D., Munoz, G., Linderman, S., & Snoek, J. (2018). Sinkhorn Networks: Learning Permutations with Sinkhorn Operator. *Workshop on Discrete Structures in Machine Learning, NeurIPS*.

[Minton et al. 1992] Minton, S., Johnston, M. D., Philips, A. B., & Laird, P. (1992). Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems. *Artificial Intelligence*, 58(1–3), 161–205.

[Nocedal & Wright 2006] Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization*, 2nd ed. Springer.

[Perron & Furnon 2023] Perron, L., & Furnon, V. (2023). OR-Tools. Google LLC. Available at https://developers.google.com/optimization.

[Powell 1969] Powell, M. J. D. (1969). A method for nonlinear constraints in minimization problems. In R. Fletcher (ed.), *Optimization*. Academic Press, London, pp. 283–298.

[Sinkhorn 1964] Sinkhorn, R. (1964). A relationship between arbitrary positive matrices and doubly stochastic matrices. *The Annals of Mathematical Statistics*, 35(2), 876–879.

[Wang et al. 2019] Wang, P.-W., Donti, P. L., Wilder, B., & Kolter, J. Z. (2019). SATNet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 6545–6554.